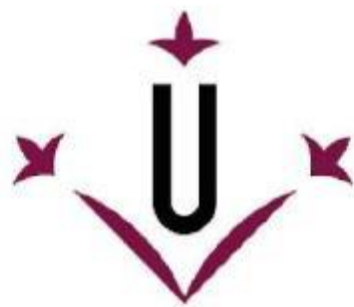




Universitat de Lleida

PROJECTE PACMAN

INTEL·LIGÈNCIA ARTIFICIAL - PRA1

Abraham Ruiz Mosteirín – Míriam Rodríguez Carranza

1. Descripció d'implementacions per a cada algorisme:**a) *ALGORISME DFS: (Instruccions clau + Definició de l'alg.)***

Primerament, l'ús d'una pila “*fringe*” la qual segueix el comportament mitjançant l'exploració dels nodes abans de retrocedir. S'ha tingut en compte, el seguiment de Nodes Visitats on es guarden en una llista “*expanded*” per evitar visitar nodes repetits i així prevenir cicles infinits. La comprovació de l'Estat Inicial on verifica l'objectivitat de l'estat inicial, evitant exploracions innecessàries. La seva construcció del camí on té la seqüència d'accions per cada node, facilitant el retorn del camí correcte quan s'arriba al “*goal*” i el retorn del camí en cas de no solució: Sinó, es retorna una llista buida ([]).

b) *ALGORISME BFS: (Instruccions clau + Definició de l'alg.)*

L'única diferència respecte el DFS es que en comptes de pila utilitza cua, “*fringe*”. El seguiment de nodes visitats fem servir una llista “*expanded*” per evitar visitar nodes duplicats, evitant així cicles i augmentant l'eficiència. Si l'estat no és objectiu, retornarà la llista buida. Per la resta d'aspectes molt similars a DFS.

c) *ALGORISME UCS: (Instruccions clau + Definició de l'alg.)*

L' UCS, s'ha implementat amb ús d'una cua amb prioritat “*fringe*” per assegurar que els nodes amb menor cost acumulat s'explorin primer. El seu seguiment de Nodes Visitats es guarden a la llista *expanded* evitant exploracions duplicades millorant l'eficiència i prevenció de generació de cicles. Ara sí, a la comprovació de l'estat objectiu, retornarà la seqüència d'accions fins a ell. La construcció de camí amb costos, guarda a cada node les accions acumulades, i el cost de cada camí amb el `problem.getCostOfActions(new_actions)`. L'actualització de nodes en cas de tenir un cost més baix, actualitza a la cua amb prioritat, garantint el camí mínim com a primera solució.

d) ALGORISME A*: (Instruccions clau + Definició de l'alg.)

Per a la implementació de A*, aquestes són les decisions principals: Ús d'una Cua amb Prioritat: A* utilitza diccionaris per a explorar els nodes amb el cost combinat més baix, calculant el cost com la suma del cost acumulat i l'heurística. El seguiment de nodes visitats, els quals tenen nodes explorats es guarden a “expanded” per evitar repetir camins, millorant l'eficiència. La construcció del camí amb cost i heurística, el va acumulant per a cada node que es calcula amb `problem.getCostOfActions(new_actions)`, i s'hi afegeix el valor de l'heurística per assegurar que el node que tingui un cost total el més baix possible, que sigui el primer explorat. L'actualització de nodes en la cua on es troben els successors, es van actualitzant per a veure si tenen un cost combinat inferior, garantint així la millor búsqueda. Aquestes decisions, permeten que A* trobi el camí òptim, combinant cost real i predicció d'heurística.

2. Explicació estratègies per a cada algorisme:

Al principi, hem fet diagrames a mode d'esquema conceptual per a saber que hauria de fer el nostre programa i a arrel d'això hem decidit quina estratègia implementar per a cada part del problema més òptimament, eficient, menys costosa possible. Després, amb l'ajut dels apunts i de les explicacions donades a classe hem desenvolupat els algorismes fent que sigui el més eficient possible tenint en compte: les expansions de nodes, temps, etc. S'ha procurat complir les expectatives de la màxima optimització possible seguint sempre la lògica de cada algoritme.

3. Anàlisis heurística del Coners/Food Search amb el cas Trivial:

1.2.2: CornersAgents (A* i UCS):

Tots dos algorismes van trobar camins amb un cost total de 16, cosa que indica que troben la solució òptima.

Temps d'execució: Encara que l'execució va ser ràpida, aquesta mètrica podria ser més rellevant a "layouts".

Nodes Expandits: L'UCS, va expandir 692 nodes. A* va expandir només 189 nodes. Conclusió que podem treure? A* va ser significativament més eficient en termes d'expansió de nodes, expandint només el 27% dels nodes que va explorar UCS per arribar a la mateixa solució òptima.

Puntuació i taxa d'èxit: Ambdues configuracions van aconseguir una puntuació de 614 i una taxa d'èxit del 100% (Win Rate: 1/1), cosa que indica que tots dos van aconseguir completar el nivell satisfactòriament.

Conclusions Eficiència en Expansió de Nodes: A* va ser més eficient en expandir menys nodes, probablement a causa d'una heurística informada que va guiar la cerca cap a l'objectiu de manera més directa. UCS, en aquest cas, va avaluar més camins possibles sense fer servir una heurística que prioritzés la proximitat a la meta.

Eficiència General: Atès que tots dos algorismes van obtenir el mateix cost de solució i puntuació, A* es destaca com l'algorisme preferit en aquest cas pel seu menor consum de recursos, expandint menys nodes per obtenir la mateixa qualitat de solució.

Conclusió Final: Per a problemes de cerca en què és possible definir una heurística que guiï la cerca cap a la meta, A* és generalment més adequat, ja que aconsegueix una solució òptima amb menys expansió de nodes, cosa que el fa més eficient en problemes similars de cerca de camins.

1.2.3: FoodSearch (A* i UCS) en Tiny i Medium:

Eficiència en Nodes Expandits: Als “layouts”, A* va expandir menys nodes en comparació amb UCS. Aquest estalvi és notable a layouts més grans “*com bigCorners i mediumCorners*”, on l'eficiència d'A* s'incrementa en aprofitar heurístiques informades per evitar exploracions innecessàries.

Temps d'Execució: A “layouts” petits, ambdós algoritmes van completar la cerca gairebé instantàniament, mentre que a “layouts” més grans, A* va ser més ràpid a causa de la menor expansió de nodes.

Consistència: Tots dos algoritmes van arribar al mateix cost total a cada “layout”, la qual cosa indica que tots dos poden trobar solucions òptimes. Tot i això, A* ho fa de forma més eficient quan es dona suport en una heurística.

Conclusió Final: L'A* és l'algorisme preferit en termes d'eficiència general degut a la capacitat de reduir l'expansió de nodes, especialment en problemes més grans i complexos. L' UCS funciona bé per obtenir solucions òptimes ara, per la manca d'heurística el converteix en una opció menys eficient en la majoria dels casos on és possible aplicar A* amb heurístiques més adequades.